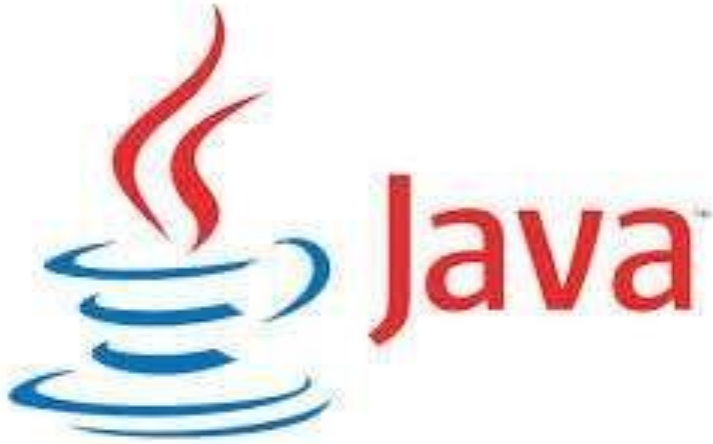


SOC2030
Application Programming in Java

Week 7 GUI 1

Dr. Andrei Dragunov

Introduction

- Graphical user interface (GUI)
  - Presents a user-friendly mechanism for interacting with an application
  - Often contains title bar, menu bar containing menus, buttons and combo boxes
  - Built from GUI components

  - https://docs.oracle.com/javase/9/

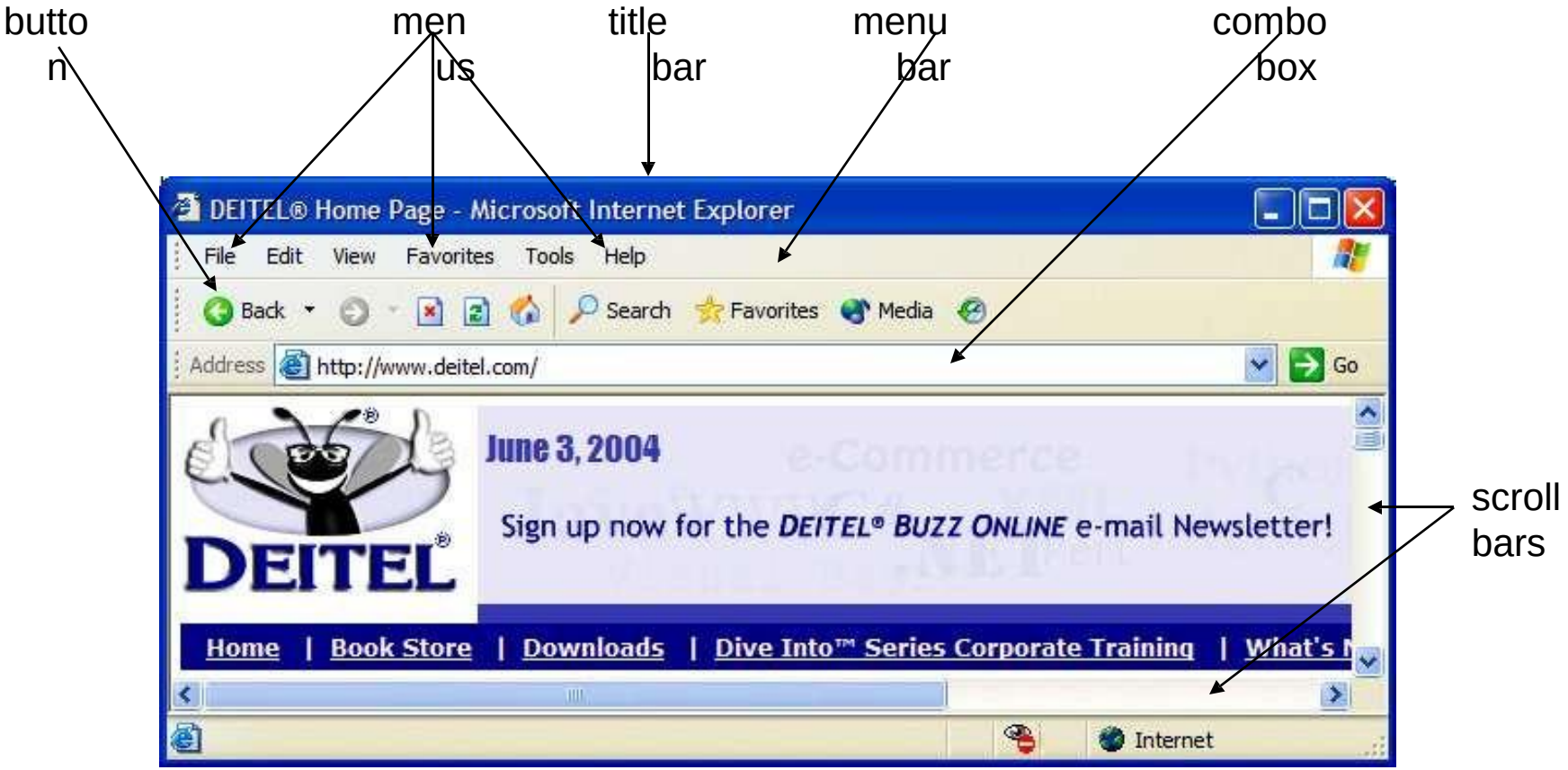


Fig. 11.1 | Internet Explorer window with GUI components.

Simple GUI-Based Input/Output with `JOptionPane`

- Dialog boxes
  - Used by applications to interact with the user
  - Provided by Java's `JOptionPane` class
    - Contains input dialogs and message dialogs

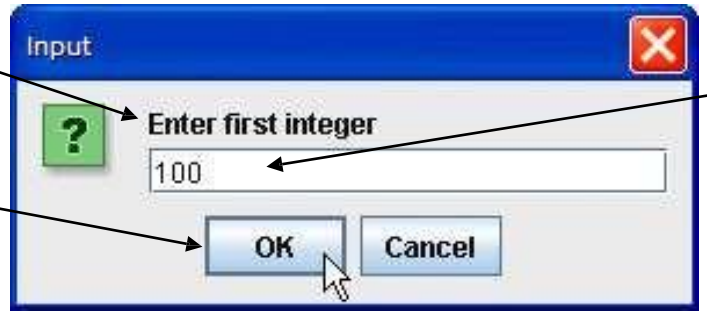
```java
// Fig. 11.2: Addition.java
// Addition program that uses JOptionPane for input and output.
import javax.swing.JOptionPane; // program uses JOptionPane

public class Addition
{
   public static void main( String args[] )
   {
      // obtain user input from JOptionPane input dialogs
      String firstNumber =
         JOptionPane.showInputDialog( "Enter first
      String secondNumber =
          JOptionPane.showInputDialog( "Enter second integer" );

      // convert String inputs to int values for use in a calculation
      int number1 = Integer.parseInt( firstNumber );
      int number2 = Integer.parseInt( secondNumber );

      int sum = number1 + number2; // add numbers

      // display result in a JOptionPane message dia
      JOptionPane.showMessageDialog( null, "The sum is " + sum,
         "Sum of Two Integers", JOptionPane.PLAIN_MESSAGE );
   } // end method main
} // end class Addition
```

Show input dialog to receive first integer

Show input dialog to receive second integer

Show message dialog to output sum to user

Input dialog displayed by lines 10–11

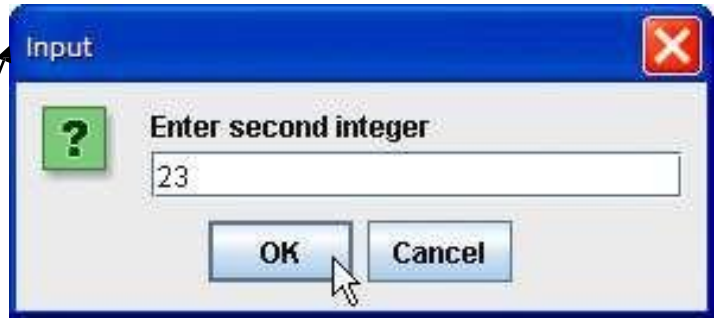


Prompt to the user

When the user clicks **OK**, `showInputDialog` returns to the program the **100** typed by the user as a `String`. The program must convert the `String` to an `int`

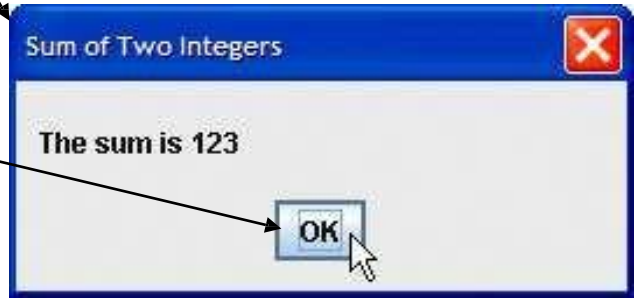
Text field in which the user types a value

Input dialog displayed by lines 12—13



title bar

Message dialog displayed by lines 22—23

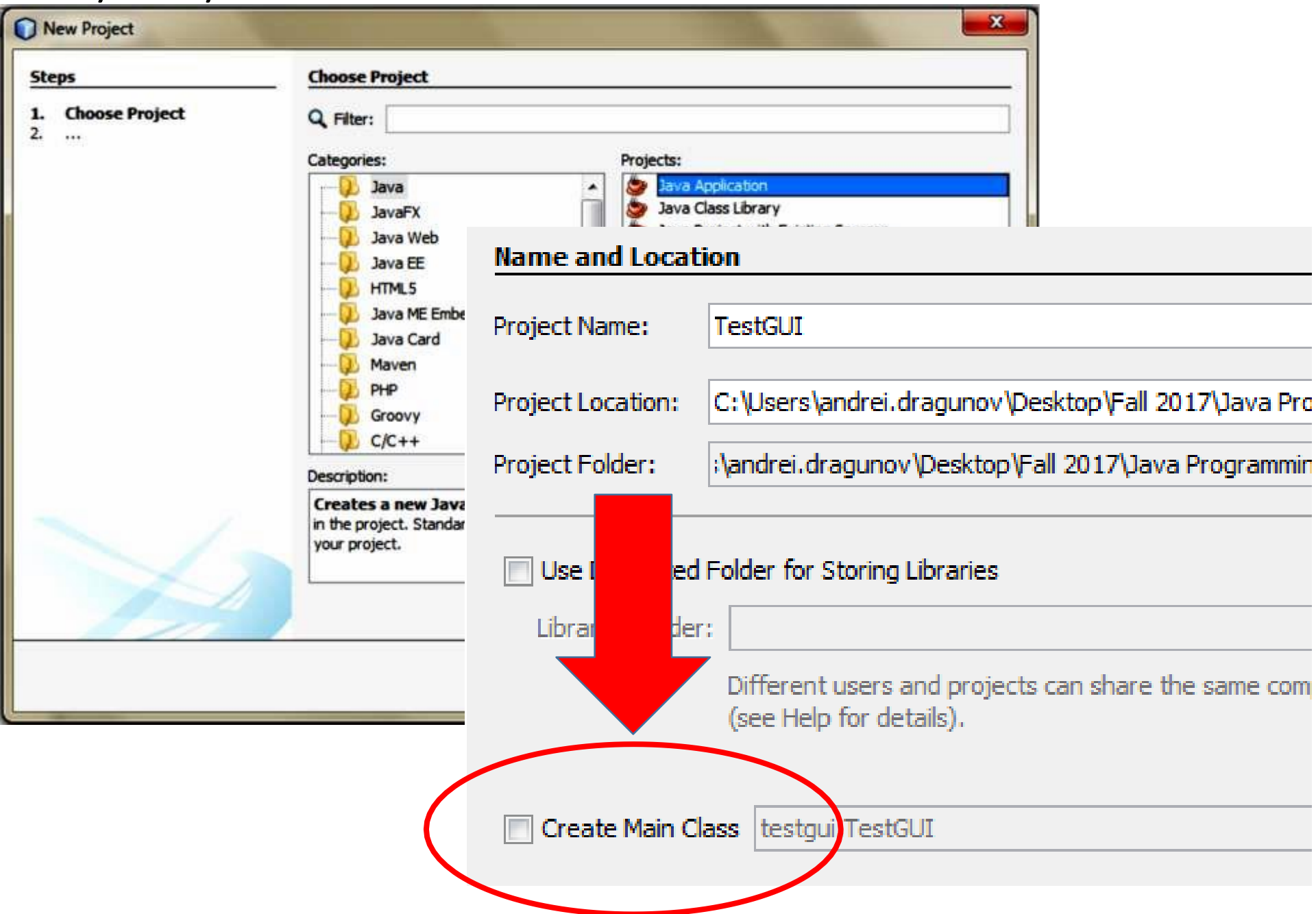


When the user clicks **OK**, the message dialog is dismissed (removed from the screen)

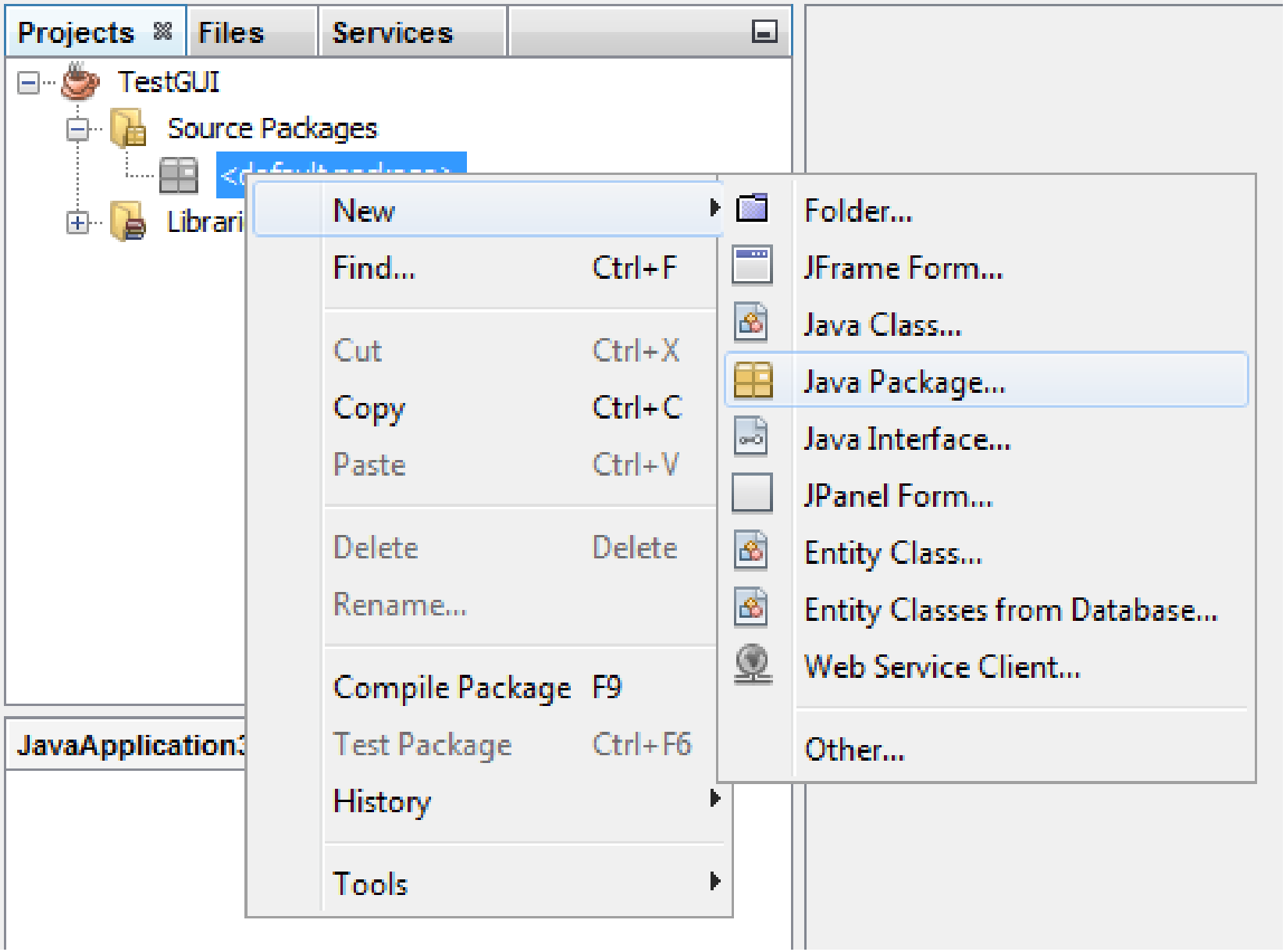
`JOptionPane` `static` constants for message dialogs.

Message dialog type	Icon	Description
`ERROR_MESSAGE`		A dialog that indicates an error to the user.
`INFORMATION_MESSAGE`		A dialog with an informational message to the user.
`WARNING_MESSAGE`		A dialog warning the user of a potential problem.
`QUESTION_MESSAGE`		A dialog that poses a question to the user. This dialog normally requires a response, such as clicking a `Yes` or a `No` button.
`PLAIN_MESSAGE`	no icon	A dialog that contains a message, but no icon.

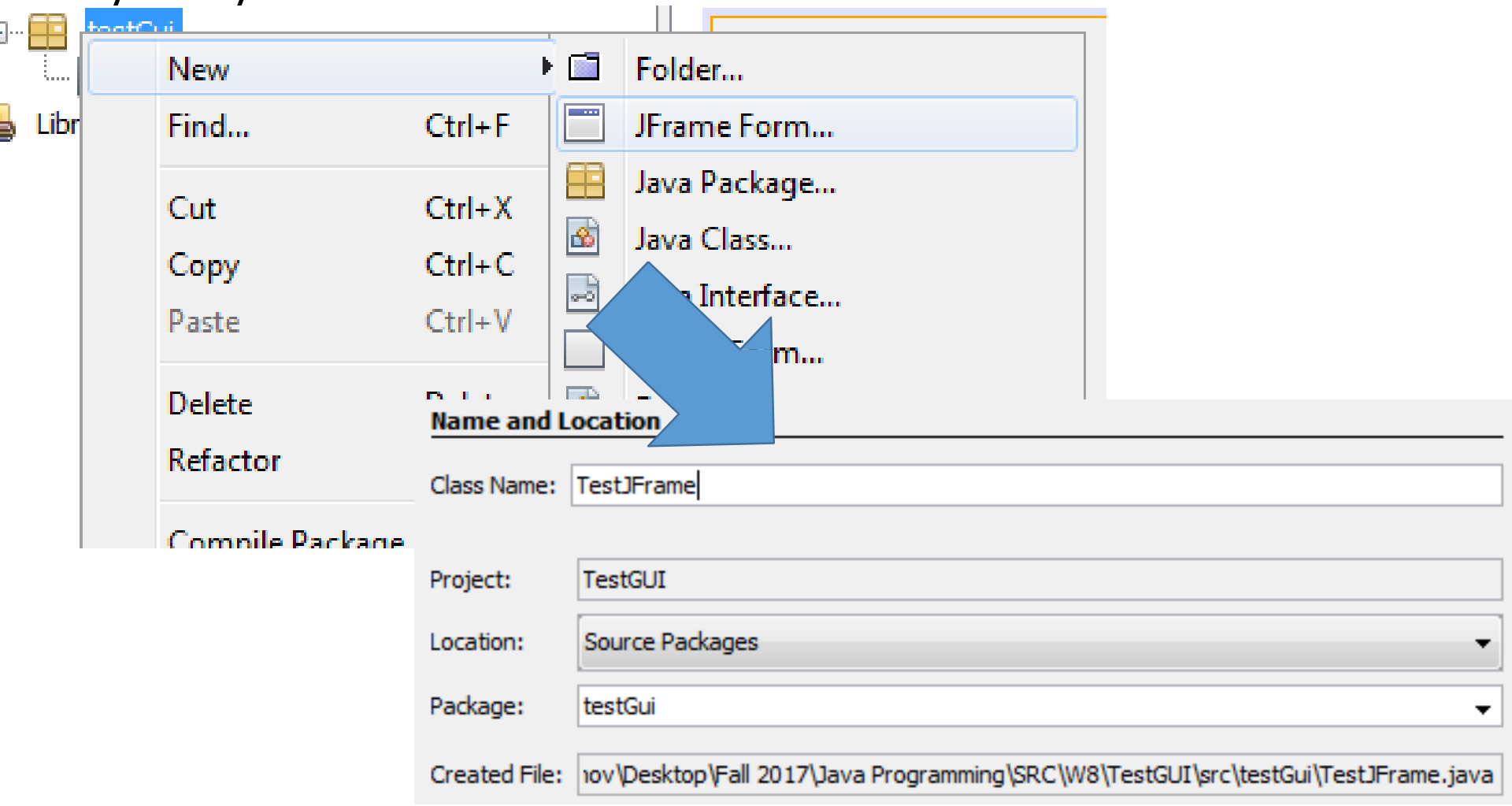
Easy way to START!



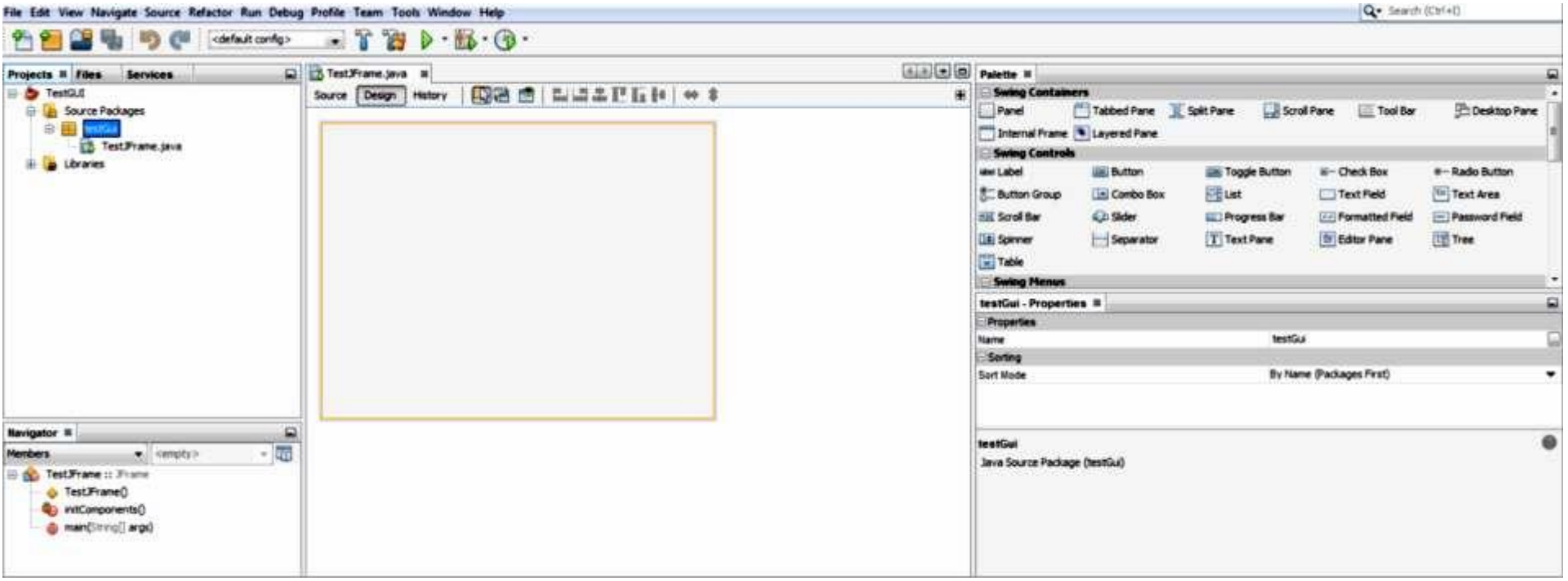
Easy way to START!



Easy way to START!

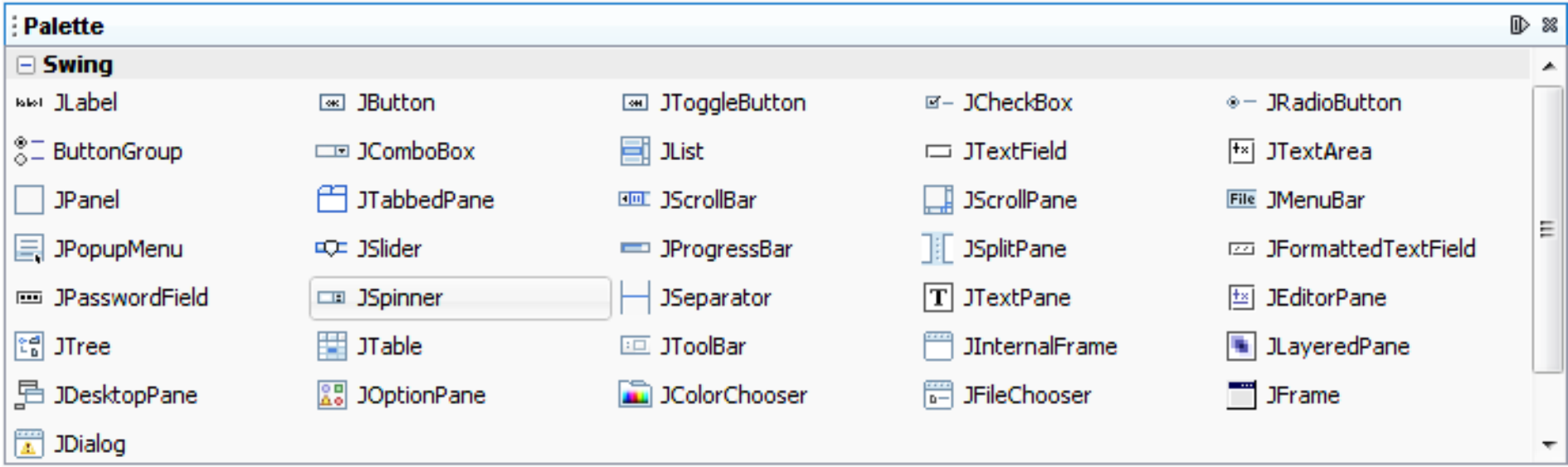


Easy way to START!



The Palette

- A customizable list of available components containing tabs for JFC/Swing, AWT, and JavaBeans components, as well as layout managers. In addition, you can create, remove, and rearrange the categories displayed in the Palette using the customizer.

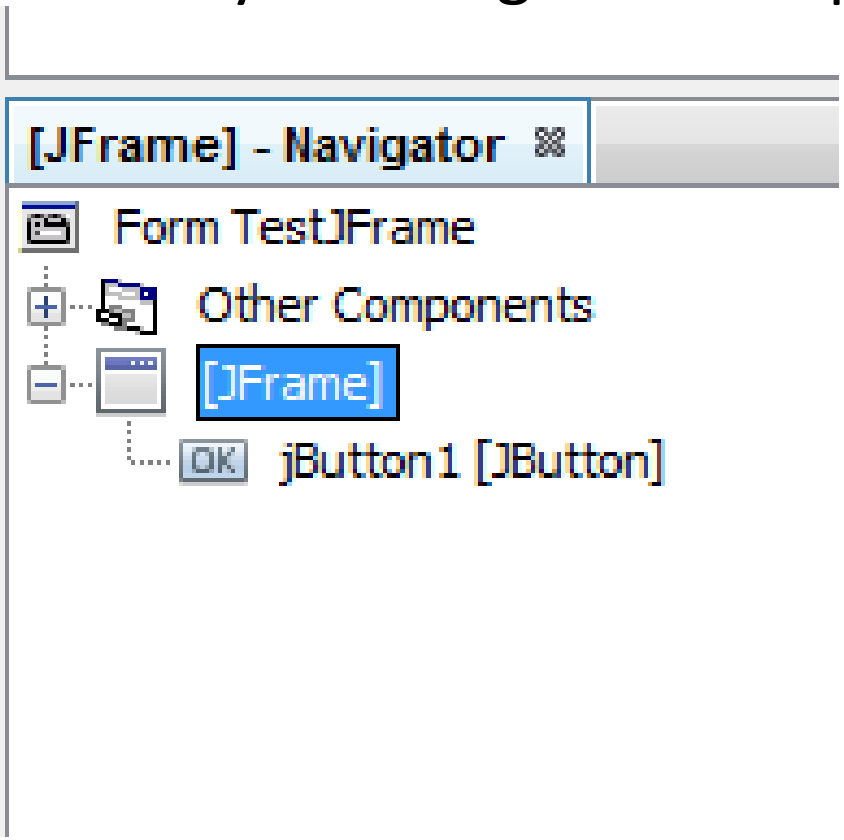


Design Area.

- The GUI Builder's primary window for creating and editing Java GUI forms. The toolbar's Source button enables you to view a class's source code, the Design button allows you to view a graphical view of the GUI components, the History button allows you to access the local history of changes of the file. The additional toolbar buttons provide convenient access to common commands, such as choosing between Selection and Connection modes, aligning components, setting component auto-resizing behavior, and previewing forms.

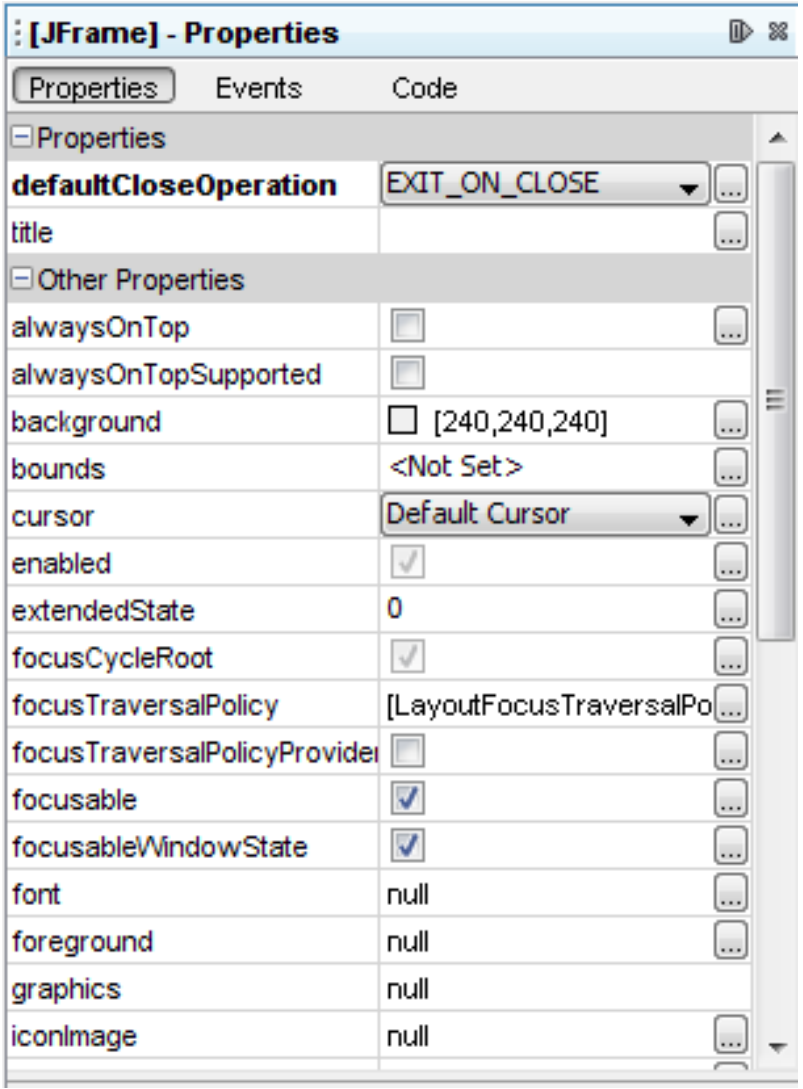
Navigator.

- Provides a representation of all the components, both visual and non-visual, in your application as a tree hierarchy. The Navigator also provides visual feedback about what component in the tree is currently being edited in the GUI Builder as well as allows you to organize components in the available panels.



Properties Window.

- Displays the properties of the component currently selected in the GUI Builder, Navigator window, Projects window, or Files window.

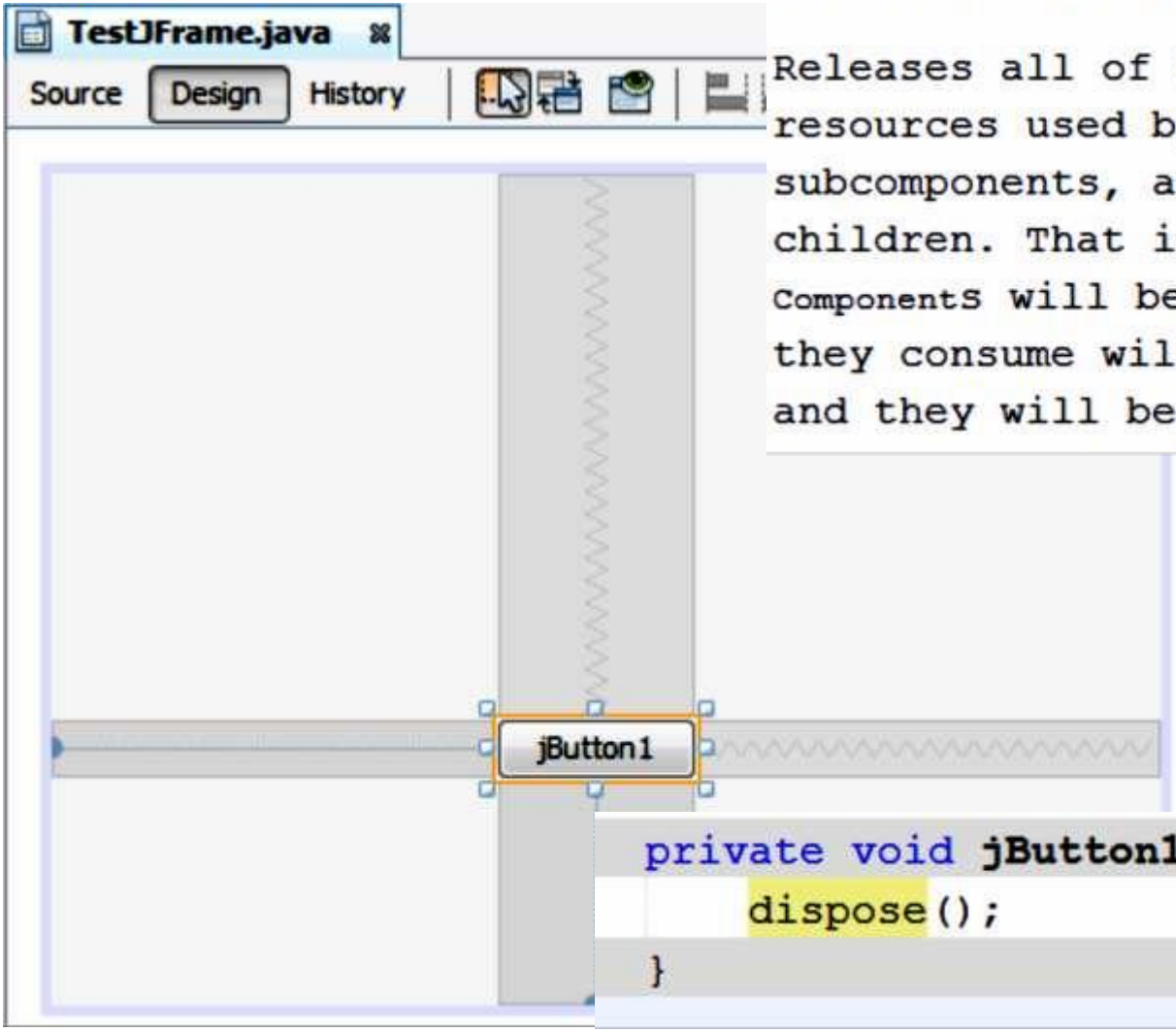


Easy way to START!

java.awt.Window

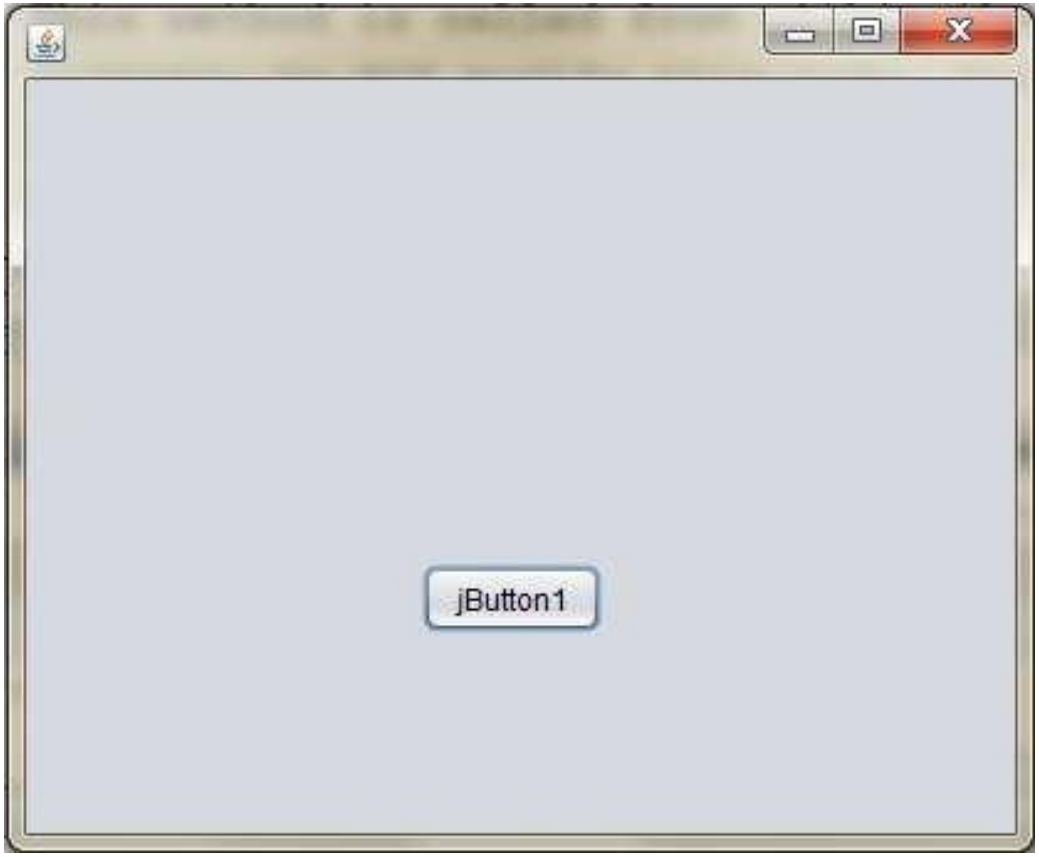
public void **dispose**()

Releases all of the native screen resources used by this Window, its subcomponents, and all of its owned children. That is, the resources for these Components will be destroyed, any memory they consume will be returned to the OS, and they will be marked as undisplayable.




```
private void jButton1ActionPerformed(java.awt.e
    dispose();          // TODO add your handling
}
```

Ready!



Key Concepts

Free Design

In the IDE's GUI Builder, you can build your forms by simply putting components where you want them as though you were using absolute positioning. The GUI Builder figures out which layout attributes are required and then generates the code for you automatically. You need not concern yourself with insets, anchors, fills, and so forth.

Key Concepts

Automatic Component Positioning (Snapping)

- As you add components to a form, the GUI Builder provides visual feedback that assists in positioning components based on your operating system's look and feel. The GUI Builder provides helpful inline hints and other visual feedback regarding where components should be placed on your form, automatically snapping components into position along guidelines. It makes these suggestions based on the positions of the components that have already been placed in the form, while allowing the padding to remain flexible such that different target look and feels render properly at runtime.

Key Concepts

Visual Feedback

- The GUI Builder also provides visual feedback regarding component anchoring and chaining relationships. These indicators enable you to quickly identify the various positioning relationships and component pinning behavior that affect the way your GUI will both appear and behave at runtime. This speeds the GUI design process, enabling you to quickly create professional-looking visual interfaces that work.

Generated code

```
@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jButton1 = new javax.swing.JButton();

    setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

    jButton1.setText("jButton1");
    jButton1.addActionListener(new java.awt.event.ActionListener() {...5
```

Overview of Swing Components

- Swing GUI components
  - Declared in package `javax.swing`
  - Most are pure Java components
  - Part of the Java Foundation Classes (JFC)

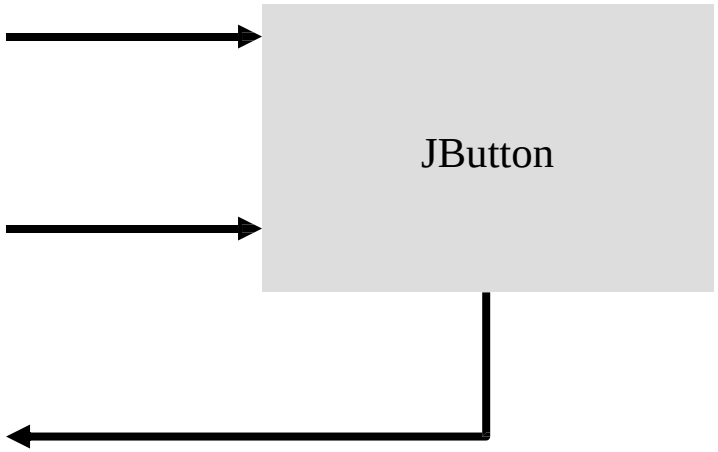
Some basic GUI components.

Component	Description
`JLabel`	**Displays uneditable text or icons.**
`JTextField`	**Enables user to enter data from the keyboard. Can also be used to display editable or uneditable text.**
`JButton`	**Triggers an event when clicked with the mouse.**
`JCheckBox`	**Specifies an option that can be selected or not selected.**
`JComboBox`	**Provides a drop-down list of items from which the user can make a selection by clicking an item or possibly by typing into the box.**
`JList`	**Provides a list of items from which the user can make a selection by clicking on any item in the list. Multiple elements can be selected.**
`JPanel`	**Provides an area in which components can be placed and organized. Can also be used as a drawing area for graphics.**

GUI Component API

- Java: GUI component = class

- Properties
- Methods
- Events

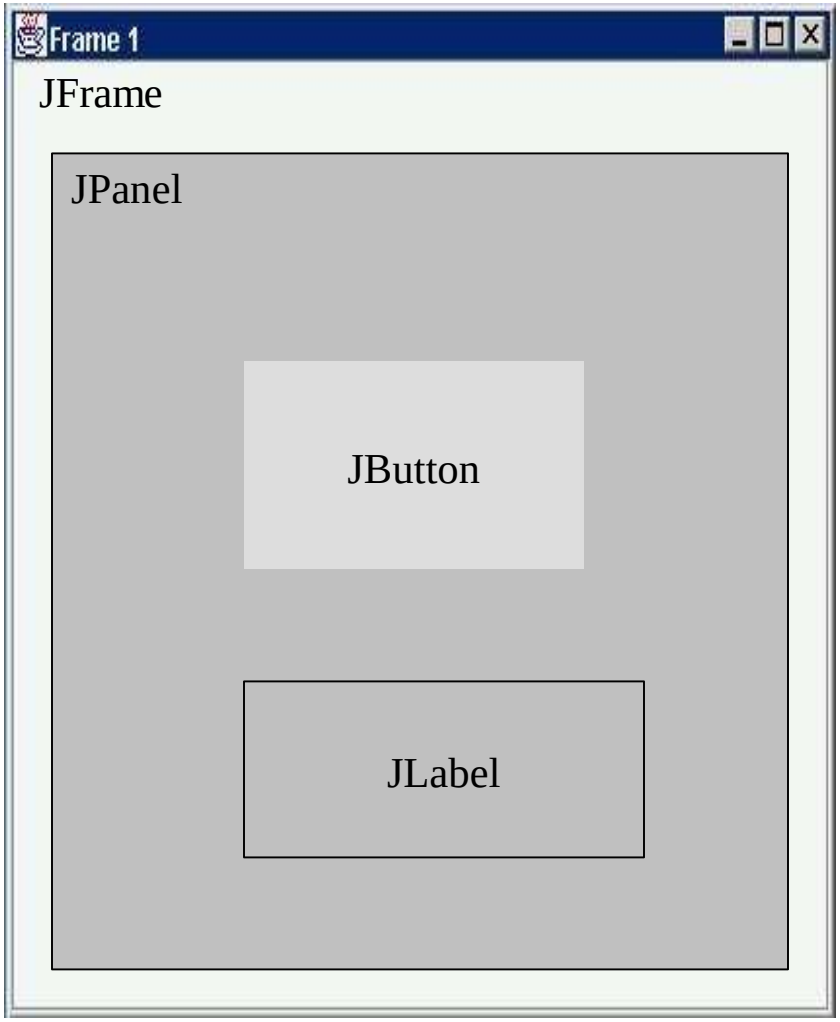


Using a GUI Component

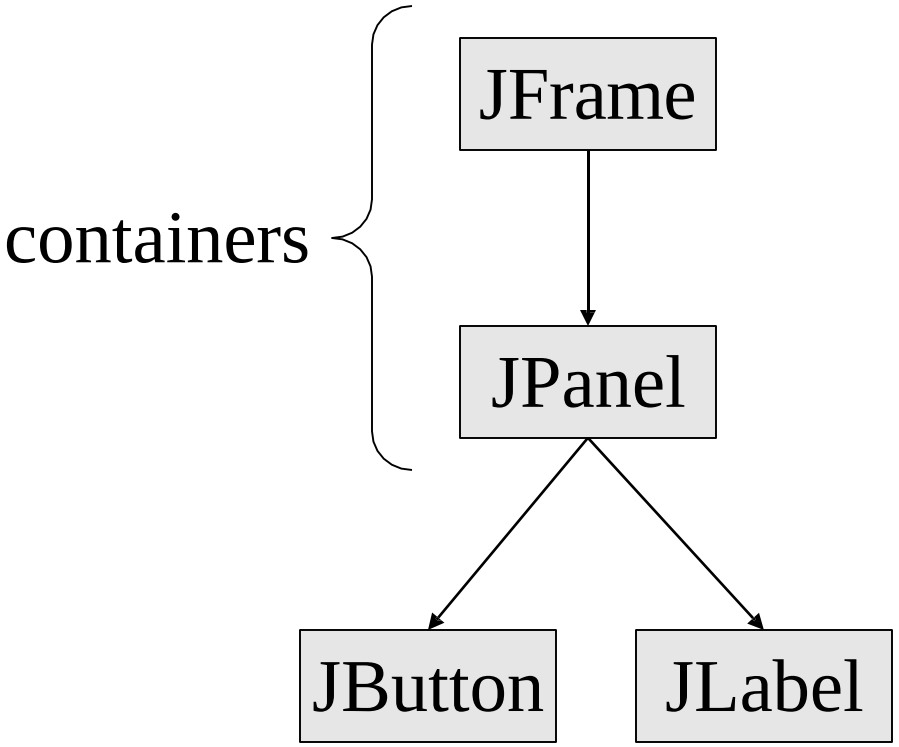
1. Create it
   - Instantiate object: b = new JButton(“press me”);
2. Configure it
   - Properties: b.text = “press me”; [avoided in java]
   - Methods: b.setText(“press me”);
3. Add it
   - panel.add(b);
4. Listen to it
   - Events: Listeners

Anatomy of an Application GUI

GUI



Internal structure

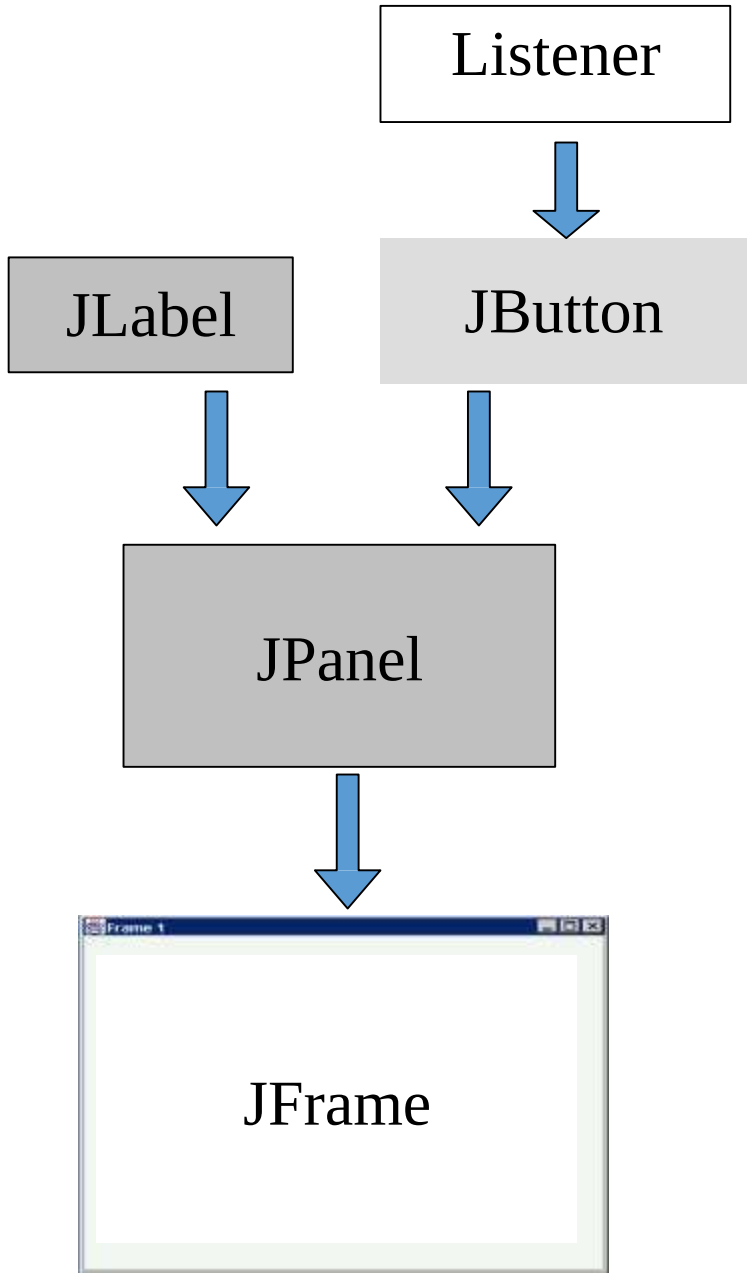


Using a GUI Component

1. Create it
2. Configure it
3. Add children (if container)
4. Add to parent (if not JFrame)
5. Listen to it

Build from bottom up

- Create:
  - Frame
  - Panel
  - Components
  - Listeners
- Add: (bottom up)
  - listeners into components
  - components into panel
  - panel into frame



Swing vs. AWT

- Abstract Window Toolkit (AWT)
  - Precursor to Swing
  - Declared in package `java.awt`
  - Does not provide consistent, cross-platform look-and-feel

Swing

- Swing components are implemented in Java, so they are more portable and flexible than the original Java GUI components from package `java.awt`, which were based on the GUI components of the underlying platform. For this reason, Swing GUI components are generally preferred.

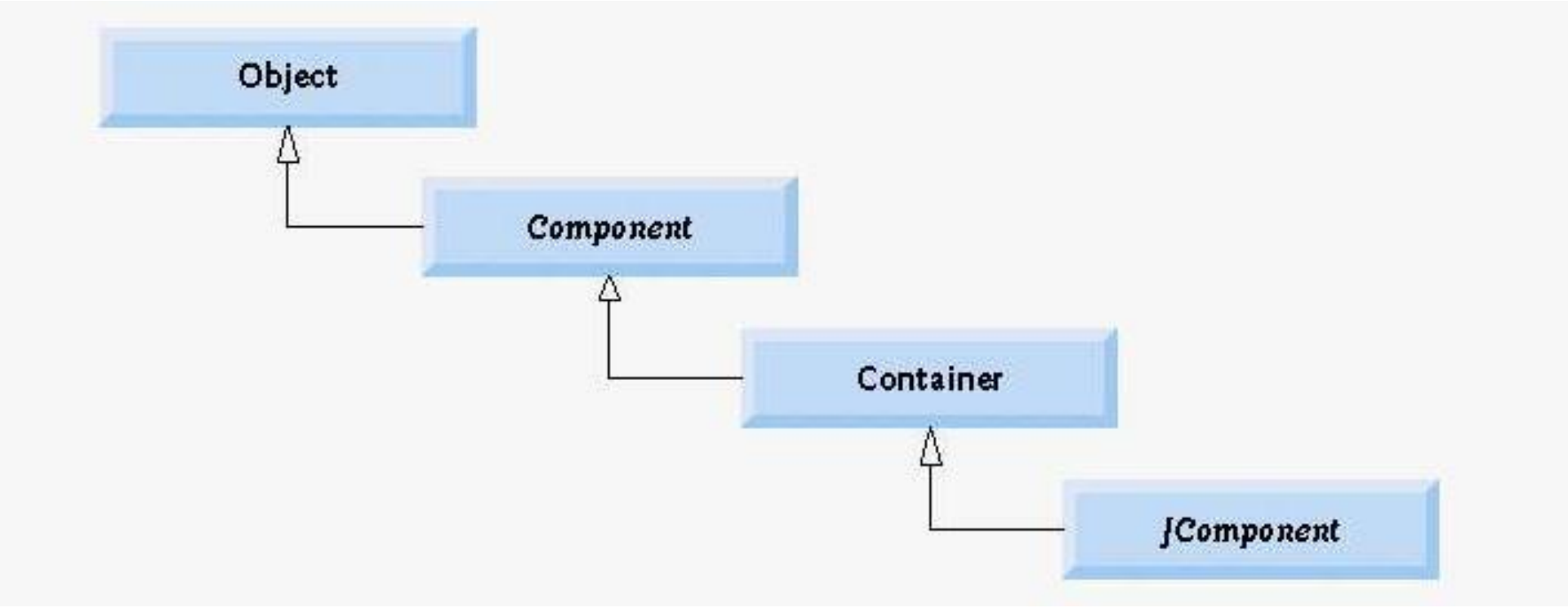
Lightweight vs. Heavyweight GUI Components

- Lightweight components
  - Not tied directly to GUI components supported by underlying platform
- Heavyweight components
  - Tied directly to the local platform
  - AWT components
  - Some Swing components

Superclasses of Swing's Lightweight GUI Components

- Class `Component` (package `java.awt`)
  - Subclass of `Object`
  - Declares many behaviors and attributes common to GUI components
- Class `Container` (package `java.awt`)
  - Subclass of `Component`
  - Organizes `Component`s
- Class `JComponent` (package `javax.swing`)
  - Subclass of `Container`
  - Superclass of all lightweight Swing components

Common superclasses of many of the Swing components.

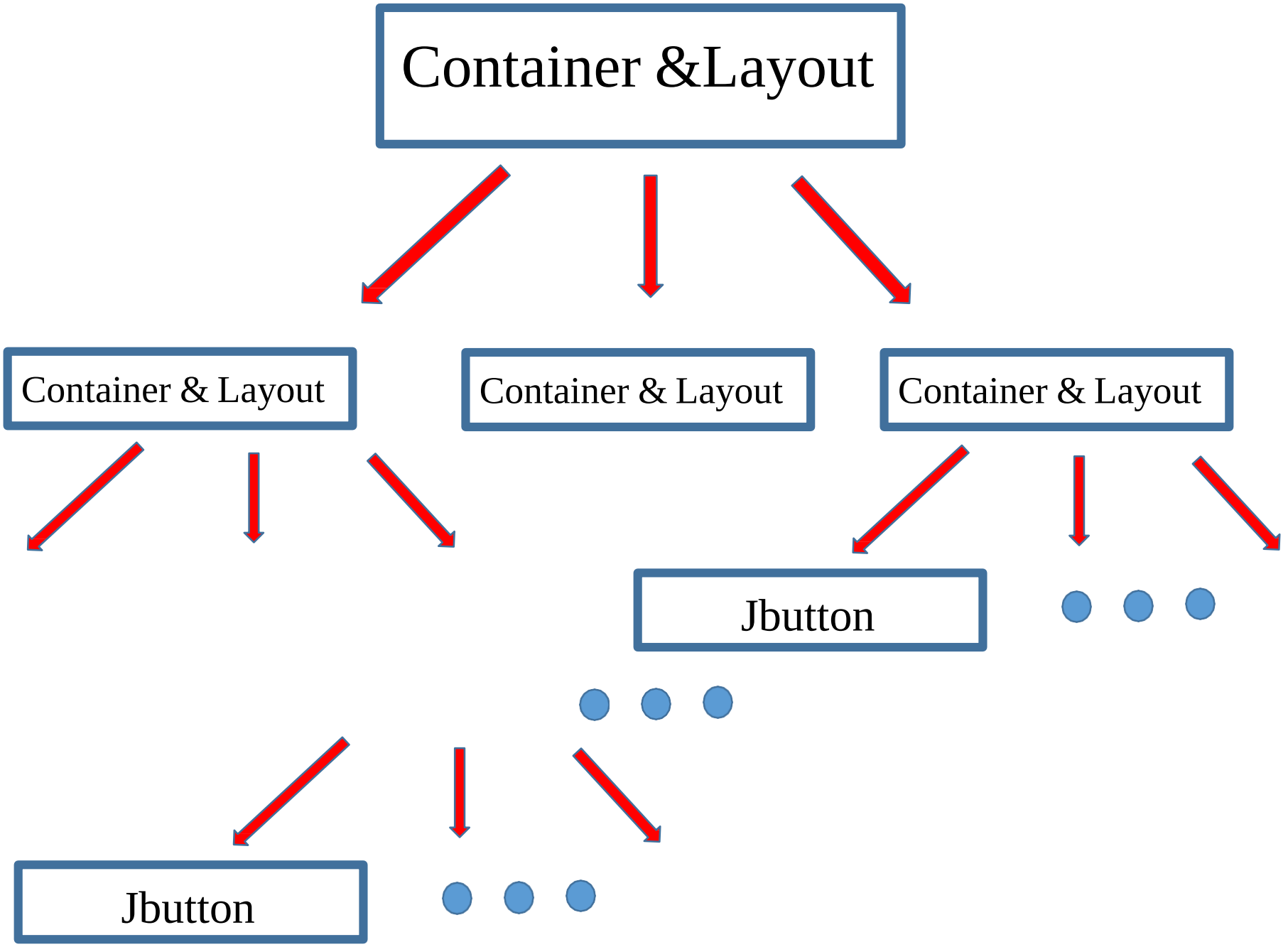


Superclasses of Swing's Lightweight GUI Components

- Common lightweight component features
  - Pluggable look-and-feel to customize the appearance of components
  - Shortcut keys (called mnemonics)

```
jButton1.setMnemonic('c');
```

  - Common event-handling capabilities
  - Brief description of component's purpose (called tool tips)
  - Support for localization


Container &Layout
Container & Layout
Container & Layout
Container & Layout
Jbutton
Jbutton

Specifying the Layout

- Laying out containers
  - Determines where components are placed in the container
  - Done in Java with layout managers
    - One of which is class `FlowLayout`
  - Set with the `setLayout` method of class `JFrame`
    - BorderLayout
    - BoxLayout
    - CardLayout
    - FlowLayout
    - GridBagLayout
    - GroupLayout
    - SpringLayout
    - GridLayout

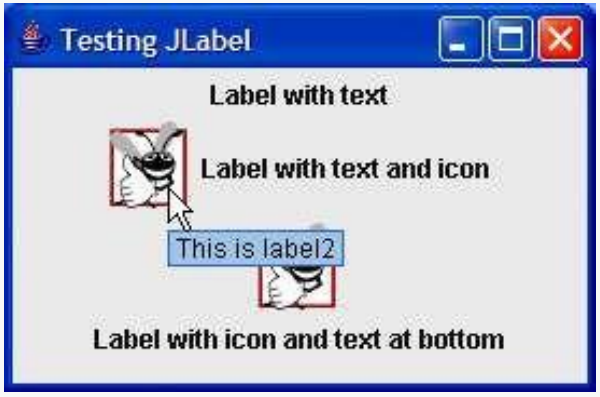
```
1  // Fig. 11.6: LabelFrame.java
2  // Demonstrating the JLabel class.
3  import java.awt.FlowLayout; // specifies how components are arranged
4  import javax.swing.JFrame; // provides basic window features
5  import javax.swing.JLabel; // displays text and images
6  import javax.swing.SwingConstants; // common constants used with Swing
7  import javax.swing.Icon; // interface used to manipulate images
8  import javax.swing.ImageIcon; // loads images
9
10 public class LabelFrame extends JFrame
11 {
12    private JLabel label1; // JLabel with just text
13    private JLabel label2; // JLabel constructed with text and icon
14    private JLabel label3; // JLabel with added text and icon
15
16    // LabelFrame constructor adds JLabels to JFrame
17    public LabelFrame()
18    {
19       super( "Testing JLabel" );
20       setLayout( new FlowLayout() ); // set frame layout
21
22       // JLabel constructor with a string argument
23       label1 = new JLabel( "Label with text" );
24       label1.setToolTipText( "This is label1" );
25       add( label1 ); // add label1 to JFrame
26
```

```
27        // JLabel constructor with string, Icon and alignment arguments
28        Icon bug = new ImageIcon( getClass().getResource(  "bug1.gif" ) );
29        label2 = new JLabel( "Label with text and icon", bug,
30           SwingConstants.LEFT );
31        label2.setToolTipText( "This is label2" );
32        add( label2 ); // add label2 to JFrame
33
34        label3 = new JLabel(); // JLabel constructor no arguments
35        label3.setText( "Label with icon and text at bottom" );
36        label3.setIcon( bug ); // add icon to JLabel
37        label3.setHorizontalTextPosition( SwingConstants.CENTER );
38        label3.setVerticalTextPosition( SwingConstants.BOTTOM );
39        label3.setToolTipText( "This is label3" );
40        add( label3 ); // add label3 to JFrame
41     } // end LabelFrame constructor
42 } // end class LabelFrame
```

ine

elTest.java

```
1  // Fig. 11.7: LabelTest.java
2  // Testing LabelFrame.
3  import javax.swing.JFrame;
4
5  public class LabelTest
6  {
7     public static void main( String args[] )
8     {
9        LabelFrame labelFrame = new LabelFrame(); // create LabelFrame
10       labelFrame.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
11       labelFrame.setSize( 275, 180 ); // set frame size
12       labelFrame.setVisible( true ); // display frame
13    } // end main
14 } // end class LabelTest
```



Note!

- If you do not explicitly add a GUI component to a container, the GUI component will not be displayed when the container appears on the screen.

How Layout Management Works

- Layout managers basically do two things:
  - **Calculate the minimum/preferred/maximum sizes for a container.**
  - **Lay out the container's children**.
- Layout managers do this based on the provided constraints, the container's properties and on the children's minimum/preferred/maximum sizes. If a child is itself a container then its own layout manger is used to get its minimum/preferred/maximum sizes and to lay it out.

- A container can be *valid* (namely, isValid() returns true) or *invalid*. For a container to be valid, all the container's children must be laid out already and must all be valid also. The Container.validate method can be used to validate an invalid container. This method triggers the layout for the container and all the child containers down the component hierarchy and marks this container as valid.
- After a component is created it is in the invalid state by default. The Window.pack method validates the window and lays out the window's component hierarchy for the first time.

GridLayout

A GridLayout places components in a grid of cells. Each component takes all the available space within its cell, and each cell is exactly the same size. If you resize the GridLayoutDemo window, you'll see that the GridLayout changes the cell size so that the cells are as large as possible, given the space available to the container.

How to Use GridLayout

The GridLayout API

The following table lists constructors of the GridLayout class that specify the number of rows and columns.

Constructor	Purpose
`GridLayout(int rows, int cols)`	Creates a grid layout with the specified number of rows and columns. All components in the layout are given equal size. One, but not both, of `rows` and `cols` can be zero, which means that any number of objects can be placed in a row or in a column.
`GridLayout(int rows, int cols, int hgap, int vgap)`	Creates a grid layout with the specified number of rows and columns. In addition, the horizontal and vertical gaps are set to the specified values. Horizontal gaps are places between each of columns. Vertical gaps are placed between each of the rows.

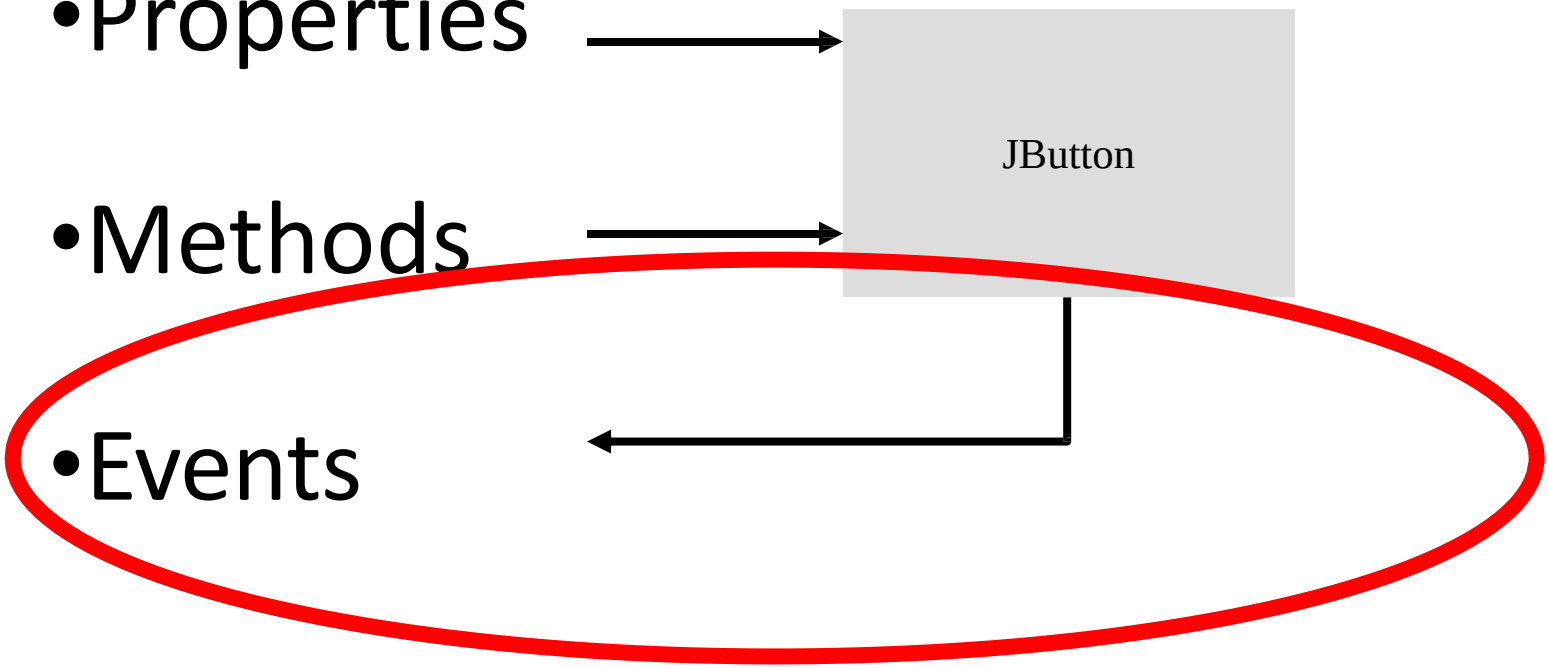
How to Use Various Layout Managers

- https://docs.oracle.com/javase/tutorial/uiswing/layout/layoutlist.html

Event-Driven Programming

- Java: GUI component = class

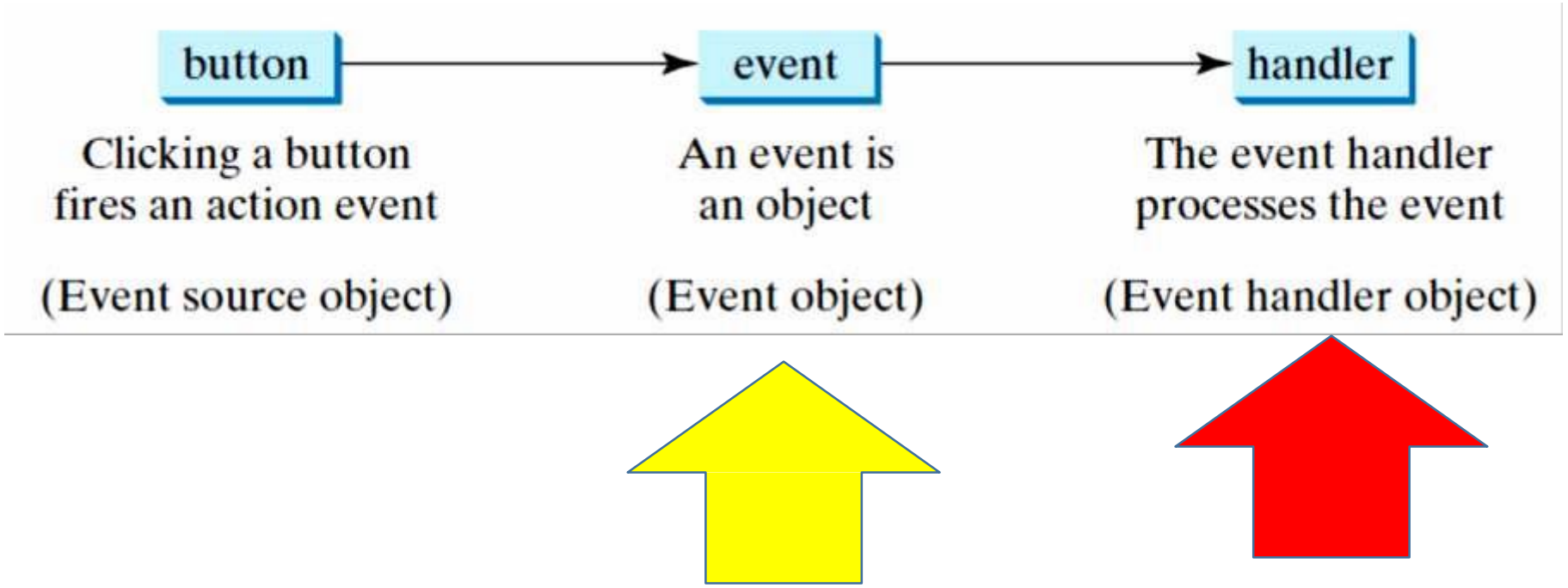
- Properties
- Methods
- Events



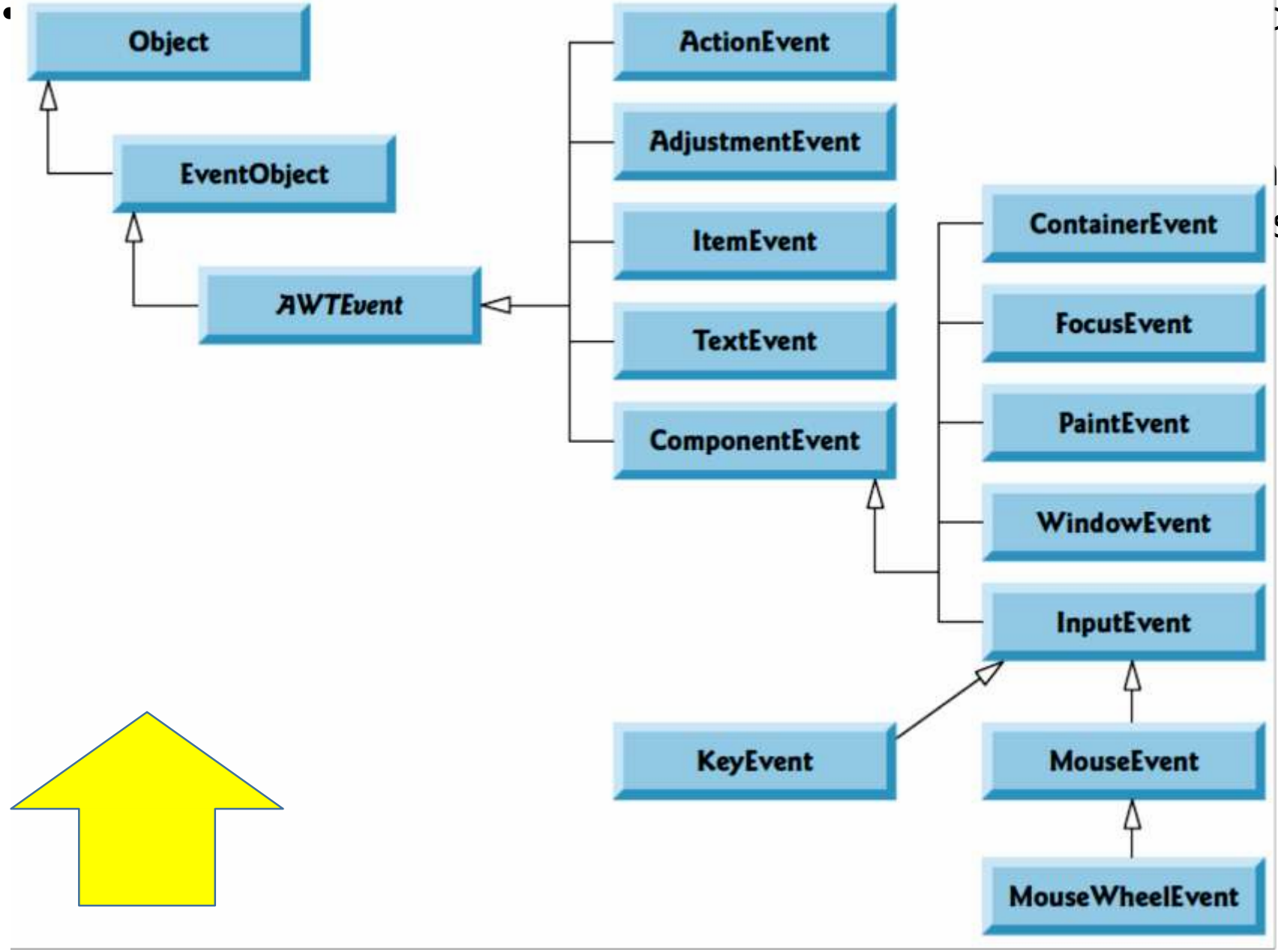
Event-Driven Programming

To respond to a button click, you need to write the code to process the button-clicking action. **The button is an *event source object*—where the action originates.**

You need to create an object capable of handling the action event on a button. This object is called an *event handler*, as shown in Figure



Common GUI Event Types and Listener Interfaces



- Not all objects can be handlers for an action event. To be a handler of an action event, two requirements must be met:

1.The object must be an instance of the **EventHandler<T extends Event>** interface. This interface defines the common behavior for all handlers. **<T extends Event>** denotes that **T** is a generic type that is a subtype of **Event**.

2.The **EventHandler** object **handler** must be registered with the event source object using the method **source.setOnAction(handler)**.

- The **EventHandler<ActionEvent>** interface contains the **handle(ActionEvent)** method for processing the action event. Your handler class must override this method to respond to the event.

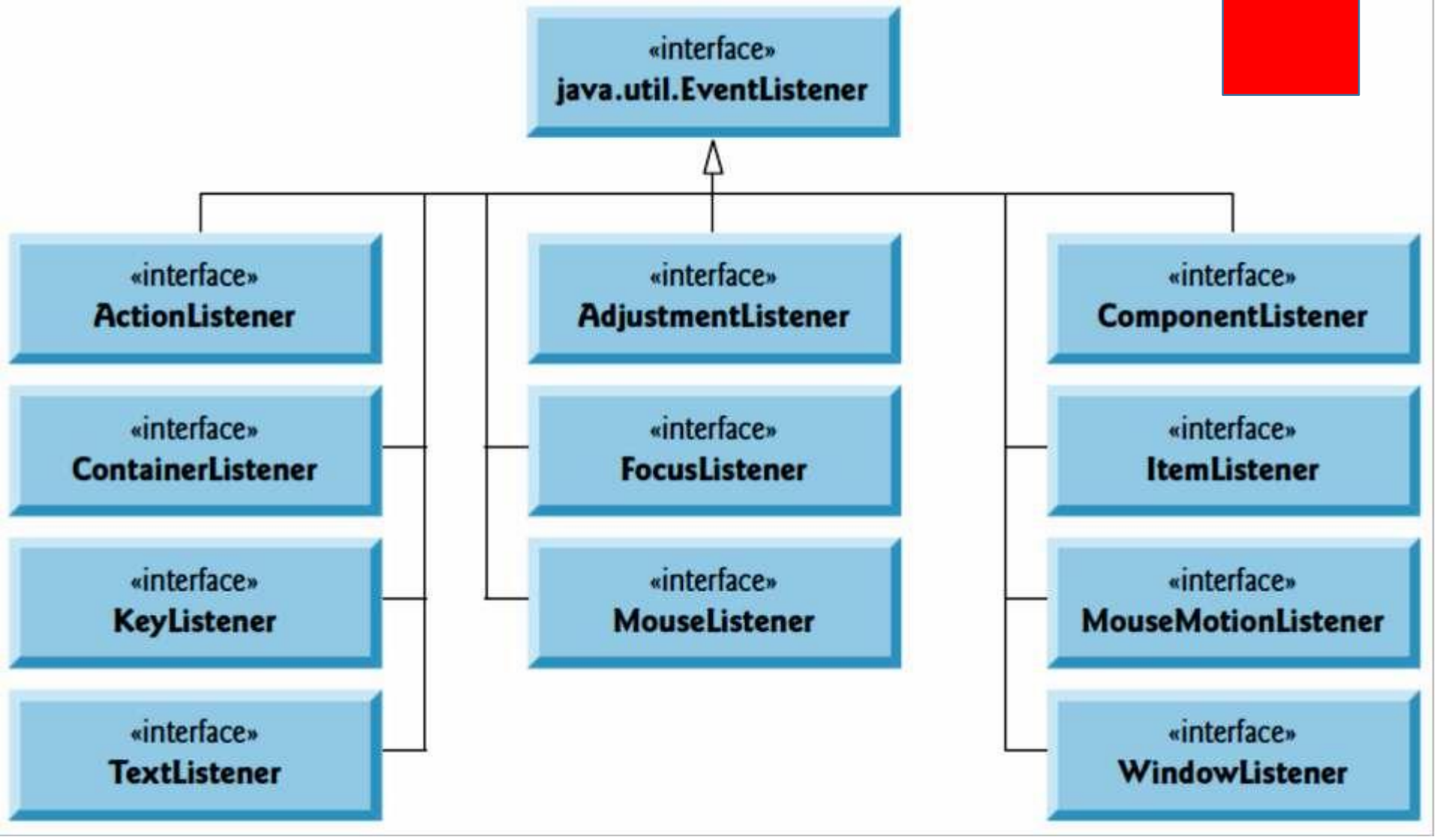
- For each event-object type, there's typically a corresponding event-listener interface.
- An event listener for a GUI event is an object of a class that implements one or more of the event-listener interfaces from packages java.awt.event and javax.swing.event.
- Many of the event-listener types are common to both Swing and AWT components.

Events and Event Sources

- *An event is an object created from an event source. Firing an event means to create an event and delegate the handler to handle the event.*

```
class ButtonHandler implements ActionListener {
 @Override
 public void actionPerformed(ActionEvent event){
    System.out.println("OK button clicked");
 }
 }
class LabelFrame extends JFrame
{
    public LabelFrame()
 {
    GridLayout l = new GridLayout(2,2);
    setLayout(l);
    JButton b1 = new JButton("test button");
    JButton b2 = new JButton("second button");
    add(b1);
    add(b2);
    ButtonHandler handler1 = new ButtonHandler();
    b1.addActionListener(handler1);

 }
}
```

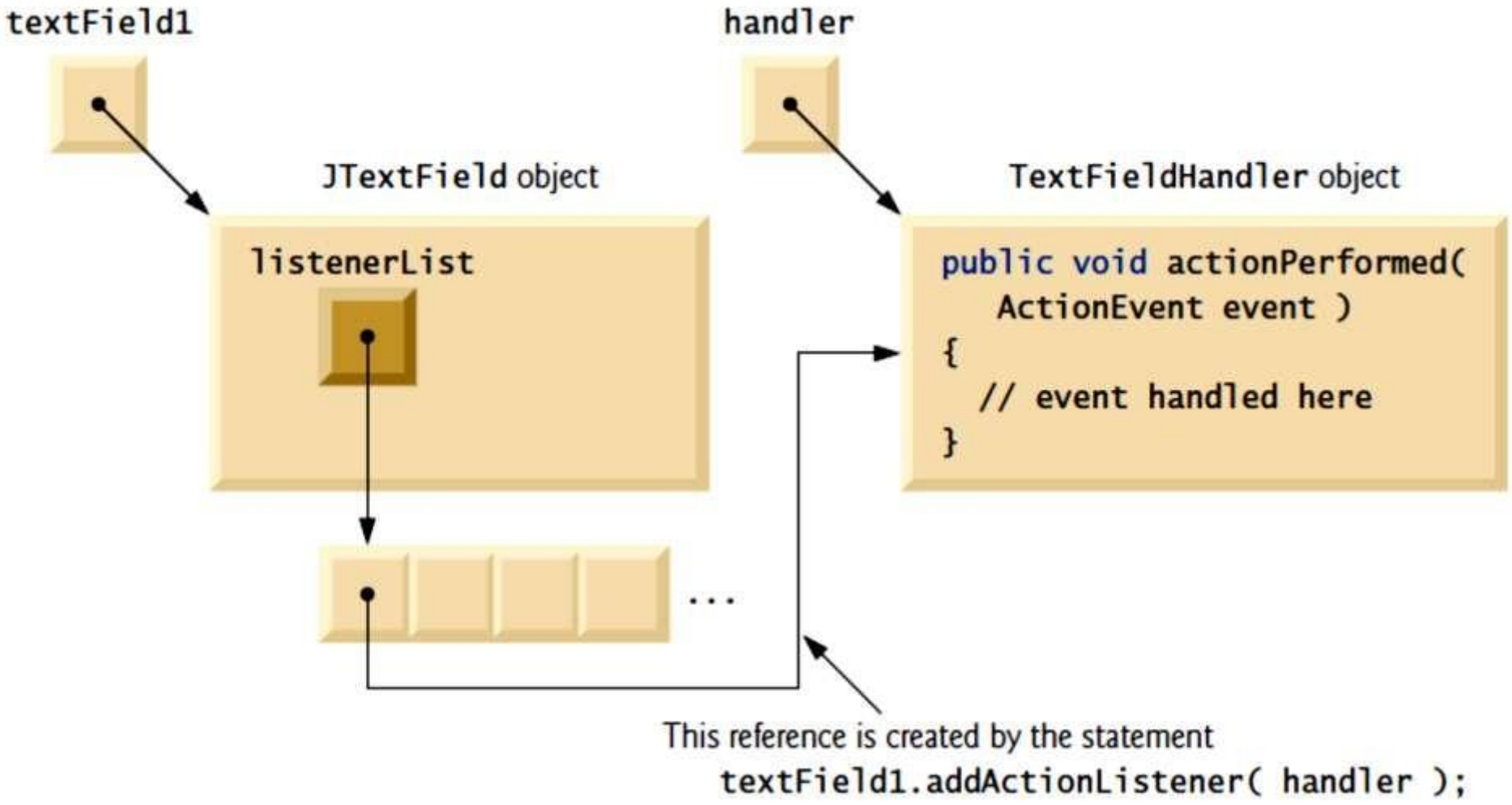
Some common event-listener interfaces of package java.awt.event



How Event Handling Works

- **1.** How did the *event handler* get *registered*?
- **2.** How does the GUI component know to call actionPerformed rather than some other event-handling method?
- ***Registering Events***
- Every JComponent has an instance variable called listenerList that refers to an object of class **EventListenerList** (package javax.swing.event). Each object of a Jcomponent subclass maintains references to its *registered listeners* in the listenerList. For simplicity, we've diagramed listenerList as an array below the JTextField object in Figure

Event registration for JTextField textField1.



- When the following statement executes

b1.addActionListener(handler2);

a new entry containing a reference to the handler2 object is placed in b1's listenerList. Although not shown in the diagram, this new entry also includes the listener's type (ActionListener).

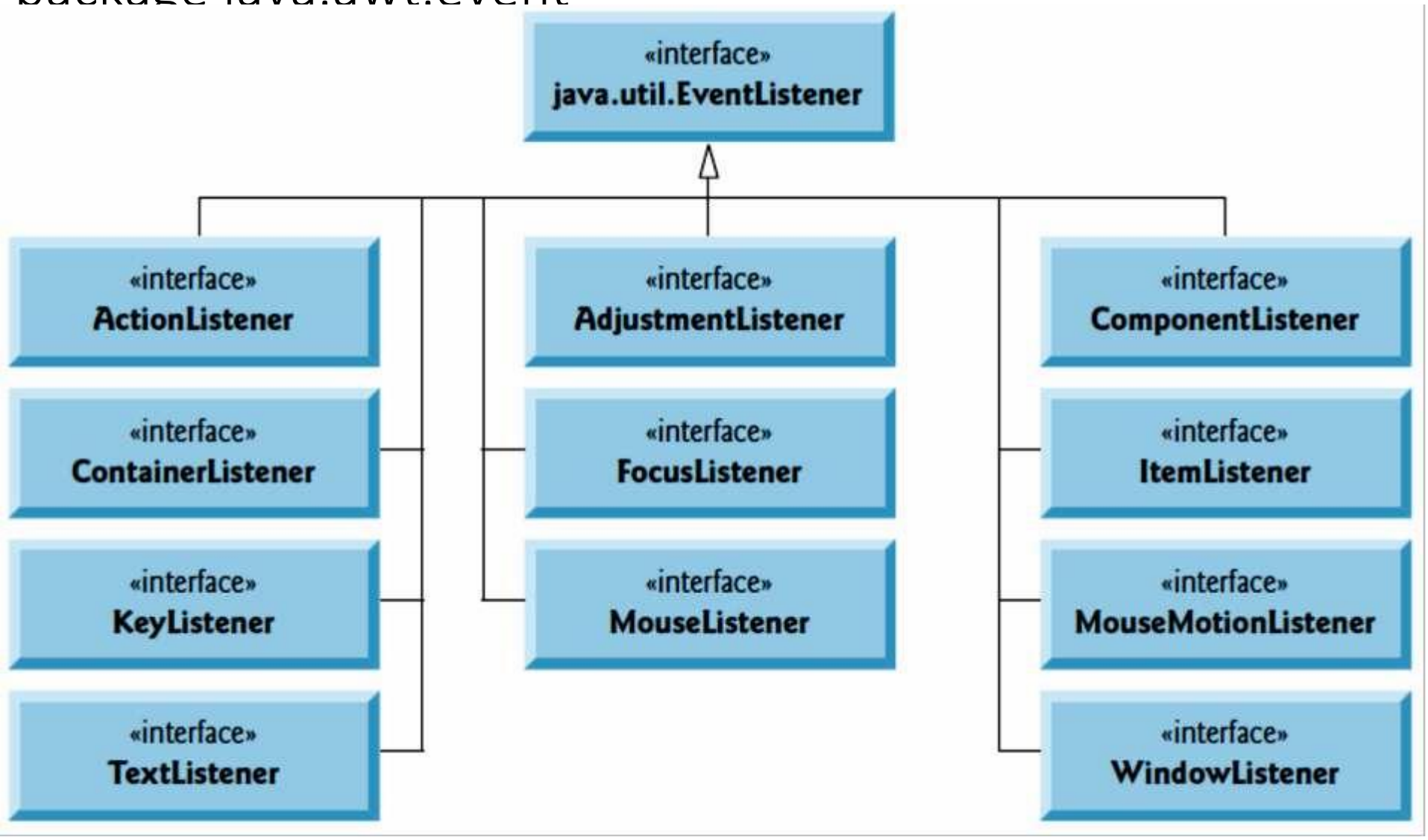
Using this mechanism, each lightweight Swing component maintains its own list of *listeners* that were *registered* to *handle* the component's *events*.

Event-Handler Invocation

- The event-listener type is important in answering the second question: How does the GUI component know to call actionPerformed rather than another method?
  Every GUI component supports several *event types*, including **mouse events**, **key events** and others. When an event occurs, the event is **dispatched** only to the *event listeners* of the appropriate type.
- Dispatching is simply the process by which the GUI component calls an event-handling method on each of its listeners that are registered for the event type that occurred.

Each *event type* has one or more corresponding *event-listener interfaces*. For example, ActionEvents are handled by ActionListeners, **MouseEvents** by **MouseListeners** and **MouseMotionListeners**, and **KeyEvents** by **KeyListeners**. When an event occurs, the GUI component receives (from the JVM) a unique ***event ID*** specifying the event type. The GUI component uses the event ID to decide the listener type to which the event should be dispatched and to decide which method to call on each listener object. For an ActionEvent, the event is dispatched to *every* registered ActionListener's actionPerformed method (the only method in interface ActionListener).

Some common event-listener interfaces of package java.awt.event

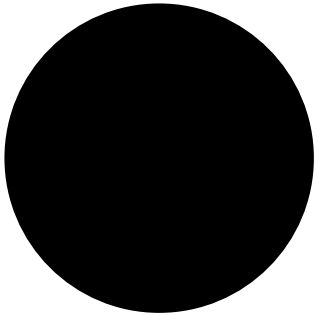


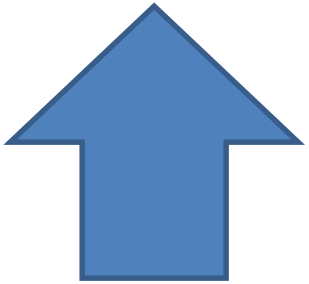
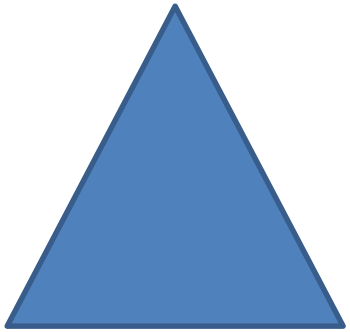
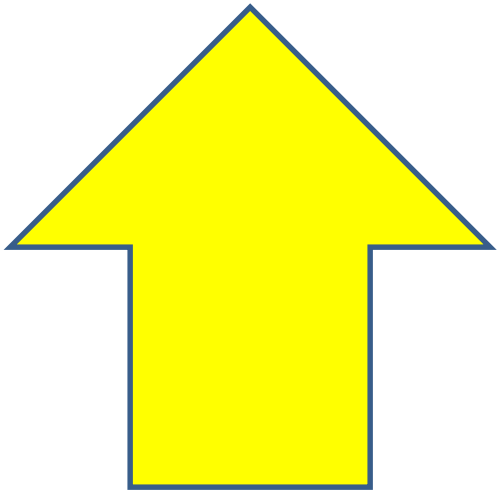
Using an Anonymous Inner Class for Event Handling

- Anonymous inner class
  - Special form of inner class
  - Declared without a name
  - Typically appears inside a method call
  - Has limited access to local variables

```
    b2.addFocusListener(new java.awt.event.FocusAdapter() {
            public void focusGained(java.awt.event.FocusEvent evt) {
                formFocusGained(evt);
            }
        });
}
    private void formFocusGained(java.awt.event.FocusEvent evt) {
        System.out.println("Second button got focus");          // TODO
    }
```

- Everything else by yourself. ;)



10.10.3 Functional Interfaces

As of Java SE 8, any interface containing only one abstract method is known as a **functional interface**. There are many such interfaces throughout the Java SE 7 APIs, and many new ones in Java SE 8. Some functional interfaces that you'll use in this book include:

- ActionListener (Chapter 12)—You'll implement this interface to define a method that's called when the user clicks a button.
- Comparator (Chapter 16)—You'll implement this interface to define a method that can compare two objects of a given type to determine whether the first object is less than, equal to or greater than the second.
- Runnable (Chapter 23)—You'll implement this interface to define a task that may be run in parallel with other parts of your program.

Functional interfaces are used extensively with Java SE 8's new lambda capabilities that we introduce in Chapter 17. In Chapter 12, you'll often implement an interface by creating a so-called anonymous inner class that implements the interface's method(s). In Java SE 8, lambdas provide a shorthand notation for creating *anonymous me*